Constructing Machine-Precision Neural Networks with Quasi-Interpolants

Anonymous Author(s)
Affiliation
Address
email

Abstract

1 Introduction

2 Related Work

3 Constructing High-Precision MLPs with Quasi-Interpolants

Our goal is to build an explicit one-hidden-layer MLP that interpolates analytic functions to arbitrarily high precision. Quasi-interpolation provides the right tool. We give an explicit construction of one-hidden-layer MLPs that realize quasi-interpolants of analytic targets. Section 3.1 builds the approximation operator and derives its coefficients, Section 3.2 realizes it as a tanh MLP, Section 3.3 derives the error–width tradeoff, and Section 3.4 summarizes the practical construction used in our experiments.

Main result (informal). For any analytic target f on $[-1, 1]$ and any target precision $\varepsilon > 0$ above a kernel-dependent threshold, we construct a one-hidden-layer MLP with $W = O(\log(1/\varepsilon))$ neurons and maximum weight magnitude $O(\log(1/\varepsilon))$ that interpolates f to accuracy ε in L^∞. The construction is explicit (Algorithm 1); for the sech^2 kernel the viable parameter regime extends to fp64 machine precision ($\varepsilon \sim 10^{-15}$), with all weights remaining within floating-point range (Table 1).

3.1 Constructing the Cardinal Function

Setup. Let $\Omega = [-1, 1]$, fix $N \in \mathbb{N}$, and define the uniform grid and scaled kernel

$$h := \frac{2}{N}, \qquad x_k := -1 + kh, \qquad K_\gamma(x) := \gamma\, K(\gamma x). \tag{1}$$

Here N is the number of grid intervals for interpolation, so there are $N + 1$ in-domain nodes $x_0, \ldots, x_N$; we later define the halo R and total width of the hidden network as $W = N + 2R + 1$. Now, A translation-invariant quasi-interpolant reconstructs f from uniform samples as

$$(Q_h f)(x) := \sum_{k \in \mathbb{Z}} f(x_k)\, L_h(x - x_k), \tag{2}$$

where the *cardinal function* L_h satisfies two conditions:

- **(i)** $\boldsymbol{L_h(jh) = \delta_{j,0}}$ for all $j \in \mathbb{Z}$ (exact interpolation at grid points);
- **(ii)** $\boldsymbol{L_h(x) = \sum_j c_j\, K_\gamma(x - jh)}$ (expansion in kernel translates, enabling MLP realization in Section 3.2).

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Stability between grid points is guaranteed by the kernel's exponential spatial decay (Assumption K4, Appendix A.1), which makes L_h a localized bump—the quasi-interpolation analog of avoiding the Runge phenomenon.

Fourier normalization. Condition **(i)** requires L_h to equal one at the origin and vanish at every other grid point. Evaluated on the lattice, this is a discrete convolution equation in the coefficient sequence (c_j). Discrete convolutions are inverted by pointwise division in Fourier space, which is the key idea behind the construction.

By Appendix B.1, condition **(i)** is equivalent to the Fourier partition-of-unity

$$\sum_{m\in\mathbb{Z}} \widehat{L}_h\Big(\omega + \tfrac{2\pi m}{h}\Big) = h \qquad \forall\, \omega. \tag{3}$$

Enforcing the kernel-expansion structure **(ii)** gives $\widehat{L}_h(\omega) = \widehat{K}_\gamma(\omega)\,\widehat{C}_h(\omega)$ with $\widehat{C}_h(\omega) := \sum_j c_j\, e^{-ijh\omega}$. Since $\widehat{C}_h$ is $2\pi/h$-periodic, substituting into (3) yields

$$\widehat{C}_h(\omega) = \frac{h}{D_h(\omega)}, \qquad D_h(\omega) := \sum_{m\in\mathbb{Z}} \widehat{K}_\gamma\Big(\omega + \tfrac{2\pi m}{h}\Big), \tag{4}$$

reducing the construction to a single division in Fourier space. Here $\widehat{C}_h$ is the *Fourier character* and D_h is the *normalizer* (the periodized kernel spectrum). The construction requires $D_h \neq 0$, which we verify for $K = \operatorname{sech}^2$ via diagonal dominance (Appendix A.1, Proposition 4).[1] Rescaling $\theta = h\omega$ gives the 2π-periodic function $\widetilde{C}_\lambda(\theta) := \widehat{C}_h(\theta/h)$. Since $\widetilde{C}_\lambda$ is 2π-periodic, its Fourier coefficients are recovered by standard inversion, and these are precisely the c_j in the kernel expansion of L_h (Appendix B.1).

From infinite to finite. To implement the operator on Ω, we introduce two truncations. *Stencil truncation*: restrict the cardinal function to $|j| \le K_c$ terms,

$$L_h^{(K_c)}(x) := \sum_{|j|\le K_c} c_j\, K_\gamma(x - jh). \tag{5}$$

Halo truncation: extend the lattice sum to $k \in \{-R, \dots, N{+}R\}$, placing R nodes on each side of Ω to stabilize boundary effects. The finite operator is

$$(Q_{h,R}^{(K_c)} f)(x) := \sum_{k=-R}^{N+R} f(x_k)\, L_h^{(K_c)}(x - x_k), \qquad x \in \Omega, \tag{6}$$

which re-indexes into a *kernel network* (Appendix B.3):

$$(Q_{h,R}^{(K_c)} f)(x) = \sum_{m=-R}^{N+R} a[m]\, K_\gamma(x - x_m), \qquad a[m] := \sum_{|j|\le K_c} c_j\, f(x_{m-j}). \tag{7}$$

3.2 MLP Realization

The kernel network (7) uses K_γ as its basis, but standard activations ψ (e.g., $\tanh$) typically do not satisfy the kernel requirements directly. Instead, a low-order derivative often does: $\operatorname{sech}^2 = \tanh'$. Assume

$$K(u) = \psi^{(r)}(u) \tag{8}$$

for some smooth ψ and order $r \in \mathbb{N}$, with K satisfying the kernel conditions (Assumptions K1–K4). By the chain rule, $K_\gamma(x - x_m) = \gamma^{-(r-1)} \frac{d^r}{dx^r}\psi(\gamma(x - x_m))$, so (7) becomes

$$(Q_{h,R}^{(K_c)} f)(x) = \frac{d^r}{dx^r}\left(\sum_{m=-R}^{N+R} \frac{a[m]}{\gamma^{r-1}}\, \psi\big(\gamma(x - x_m)\big)\right). \tag{9}$$

[1] The proofs in Appendix A use the mass-normalized profile $K_0 = \frac{1}{2}\operatorname{sech}^2$ for which $\widehat{K}_0(0) = 1$. The construction with $K = \operatorname{sech}^2 = 2K_0$ produces cardinal coefficients that are half those of the K_0 construction; all error-bound prefactors in Theorem 1 are stated for the K_0 convention and carry over with the corresponding factor.

Define the one-hidden-layer MLP

$$g_{\mathrm{MLP}}(x) := \sum_{m=-R}^{N+R} w[m]\,\psi\big(\gamma(x - x_m)\big), \qquad w[m] := \frac{a[m]}{\gamma^{r-1}}, \tag{10}$$

so that $g^{(r)}_{\mathrm{MLP}}(x) = (Q^{(K_c)}_{h,R} f)(x)$. Applying the construction to $f^{(r)}$, the MLP recovers f up to an integration polynomial:

$$\widetilde{f}(x) = p_{r-1}(x) + g_{\mathrm{MLP}}(x), \tag{11}$$

where p_{r-1} is fixed by boundary conditions. Because $K = \psi^{(r)}$, the construction is applied to $f^{(r)}$, not f directly: the kernel network approximates the r-th derivative, and the MLP recovers f by integration. For $\tanh$ ($r = 1$, $K = \mathrm{sech}^2$), this means the algorithm takes f' as input, each sech^2 unit integrates to a $\tanh$ unit with no residual scale factors, and the integration polynomial reduces to a single bias fixed by pinning $\widetilde{f}(-1) = f(-1)$. All kernel conditions (K1–K4) are verified for sech^2 in Appendix A.1.

Expressivity is not the bottleneck. Equations (10)–(11) show that one-hidden-layer tanh MLPs can exactly represent the quasi-interpolant construction. Whether machine-precision interpolation is achievable reduces to whether the correct weights can be *found*—an optimization question.

3.3 Error Analysis and Width–Precision Scaling

The dimensionless bandwidth. We introduce

$$\lambda := \gamma h.$$

Since the kernel width is on the order of $1/\gamma$ and the grid spacing is h, the ratio $1/\lambda$ is approximately the number of grid points within one kernel width. The parameter λ controls a fundamental tradeoff: large λ (narrow kernels) makes the cardinal function easy to construct but introduces *aliasing error* from spectral energy above the Nyquist rate; small λ (wide kernels) suppresses aliasing but demands large, precisely cancelling coefficients, making the construction *ill-conditioned* (prefactors in Theorem 1 diverge as $\lambda \to 0$).

Error decomposition. The total error decomposes into four exponentially decaying terms (full proof in Appendix A.1, Theorem 5).

Theorem 1 (Numerical error bound). *Let $\lambda = \gamma h$. Assume f is analytic in a strip of half-width ρ around Ω, and assume the normalizer D_h is bounded away from zero on a strip of half-width σ. Then*

$$\|f - Q^{(K_c)}_{h,R} f\|_{L^\infty(\Omega)} \le \underbrace{C_1(\lambda)\, e^{-c_1/h}}_{\text{resolution}} + \underbrace{C_2\, e^{-c_2/\lambda^p}}_{\text{aliasing}} + \underbrace{C_3(\lambda)\, e^{-c_3\lambda R}}_{\text{halo}} + \underbrace{C_4(\lambda)\, e^{-c_4\lambda K_c}}_{\text{stencil}}. \tag{12}$$

The rates $c_1, \dots, c_4$ depend on the target's analyticity ρ, the kernel's decay rate a_K, and the normalizer's strip margin σ. The prefactors C_1, C_3, C_4 diverge algebraically as $\lambda \to 0$ (explicit formulas in Appendix A.1). C_2 is independent of λ. For $K = \mathrm{sech}^2$, $p = 1$.

Width–precision scaling. To achieve target tolerance ε, we choose λ in the *viable regime*: small enough that $C_2\, e^{-c_2/\lambda^p} \le \varepsilon$, yet large enough that the prefactors remain moderate (Appendix C.1). With any λ^* in this interval, each remaining control parameter must scale logarithmically—$N = O(c_1^{-1}\log(1/\varepsilon))$ for resolution, $R = O((c_3\lambda^*)^{-1}\log(1/\varepsilon))$ for the halo, and $K_c = O((c_4\lambda^*)^{-1}\log(1/\varepsilon))$ for the stencil—giving total width $W = N + 2R + 1 = O(\log(1/\varepsilon))$.

Corollary 1 (Exponential convergence in width). *Fix a target precision ε in the viable regime and choose λ^* accordingly. There exist $A, \alpha > 0$ such that the constructed MLP with W neurons satisfies $\|f - \widetilde{f}_W\|_{L^\infty} \le A\, e^{-\alpha W}$. (Proof in Appendix C.2.)*

This matches the rate of classical polynomial interpolation for analytic targets (Trefethen, 2019), realized by a one-hidden-layer MLP with explicit weights.

Table 1: Comparison of parameter count, working-precision bit complexity, and weight magnitudes for prior MLP constructions on analytic targets.

Method	#params vs. ε	Bit complexity (working prec.)	Weight magnitudes
Cybenko (1989)	$\Theta(\varepsilon^{-1})$	$\Theta(\log(1/\varepsilon))$	$\|W\|_{\max} = \Theta\big(\varepsilon^{-1}\log(1/\varepsilon)\big)$
Mhaskar (1993)	$\widetilde{O}\big(\log(1/\varepsilon)\log\log(1/\varepsilon)\big)$	$\gtrsim \widetilde{\Theta}\big(\log(1/\varepsilon)\log\log(1/\varepsilon)\big)$	$\|W\|_{\max} = \Theta(\varepsilon^{-1/k})$ (order k)
Quasi-interpolant (ours)	$O(\log(1/\varepsilon))$	$O(\log(1/\varepsilon) + \log\log(1/\varepsilon))$	$\|W\|_{\max} = O(\log(1/\varepsilon))$

Algorithm 1 Explicit Quasi-Interpolant MLP Construction (tanh)

Input: grid size N, dimensionless bandwidth λ, halo size R, stencil half-width K_c, target f and its derivative f'.

1: **Grid.** Set $h = 2/N$, $\gamma = \lambda/h$. Build the extended grid $\{x_k = -1 + kh\}_{k=-R-K_c}^{N+R+K_c}$.

2: **Cardinal coefficients.** Solve the symmetric Toeplitz system $Tc = e_0$ for the unknowns $\{c_j\}_{|j|\le K_c}$, where $T_{k,j} = h\,K_\gamma((k-j)h)$ and equations are enforced for $|k| \le K_c$. (These are equivalently the Fourier coefficients of $\widetilde{C}_\lambda = h/\widetilde{D}_\lambda$; see Appendix B.2.)

3: **Sample f'.** Evaluate $f'(x_k)$ on the extended grid.

4: **Convolve.** Form the kernel-network coefficients $a[m] = \sum_{|j|\le K_c} c_j\, f'(x_{m-j})$ for $m \in \{-R, \dots, N+R\}$.

5: **Integrate.** The tanh MLP for f is the antiderivative of the kernel network:

$$\widetilde{f}(x) = b + \sum_{m=-R}^{N+R} a[m]\,\tanh\big(\gamma(x - x_m)\big).$$

6: **Pin boundary.** Fix the bias $b = f(-1) - \sum_m a[m]\,\tanh(\gamma(-1 - x_m))$.

Output: explicit one-hidden-layer MLP $\widetilde{f}$ with $W = N + 2R + 1$ neurons.

Weight scaling. Maintaining λ^* constant requires $\gamma = \lambda^* N/2 = O(W)$, so the maximum weight magnitude is $O(\log(1/\varepsilon))$—within fp64 range even at machine precision (Appendix C.3), unlike prior constructions where weights scale as $\Theta(\varepsilon^{-1})$ or $\Theta(\varepsilon^{-1/k})$ (Table 1).

Predictions for trained networks. The construction makes three testable predictions about what a trained MLP *should* look like to achieve high precision: (i) $\lambda = \gamma h$ should plateau as width increases, requiring γ to grow proportionally with N; (ii) the outer-layer weight magnitudes should remain $O(1)$, not grow; and (iii) the learned features should distribute approximation power uniformly rather than concentrating it in a compressible subset. In Section 4, we find that end-to-end trained MLPs violate all three: most strikingly, trained networks keep $\gamma = O(1)$ as width grows, driving $\lambda \to 0$ into the ill-conditioned regime where the error-bound prefactors diverge.

3.4 Practical Construction

The construction makes the following assumptions on the target:

(i) f and f' can be evaluated exactly (noise-free) at prescribed grid points;

(ii) f is evaluable on an extended domain beyond $[-1, 1]$, including $R + K_c$ halo points on each side;

(iii) conservative analyticity information, or safe numerical surrogates, are available for setting λ, R, and K_c.

These are strong requirements—in practice one typically has noisy samples of f at scattered points—and they are the price of a fully explicit, analyzable construction. The goal is to establish what the MLP *can* represent with the correct weights, isolating optimization as the bottleneck (Section 4).

The parameters λ, R, and K_c are numerical design choices that govern aliasing, boundary truncation, and stencil conditioning respectively. The convolution in Step 4 is numerically delicate due to cancellation among large alternating-sign coefficients; stabilization techniques, precision regimes, and full parameter guidance are in Appendix D.

4 Experiments

5 Discussion

References

A Theoretical Results

B Construction Derivations

This appendix derives the Fourier normalization construction summarized in Section 3.1. The proof of Theorem 1 (with explicit constants), Lemmas 2–4, and the kernel verification for sech^2 are in Appendix A.1.

B.1 Deriving the Cardinal Function via Fourier Normalization

We establish that the cardinal property $L_h(jh) = \delta_{j,0}$ is equivalent to (3), and show how Fourier normalization yields equation (4).

B.1.1 The cardinal property in Fourier space

Write

$$L_h(jh) = \frac{1}{2\pi} \int_{\mathbb{R}} \widehat{L}_h(\omega)\, e^{i\omega jh}\, d\omega.$$

Partition $\mathbb{R}$ into intervals of length $2\pi/h$ and substitute $\omega \mapsto \omega + 2\pi m/h$ in each:

$$L_h(jh) = \frac{1}{2\pi} \int_{-\pi/h}^{\pi/h} \underbrace{\left[\sum_{m\in\mathbb{Z}} \widehat{L}_h\Big(\omega + \tfrac{2\pi m}{h}\Big) \right]}_{=:\, S(\omega)} e^{i\omega jh}\, d\omega,$$

using $e^{i(2\pi m/h)jh} = 1$. The integral extracts the j-th Fourier coefficient of S on $[-\pi/h, \pi/h]$.

Forward. If $S(\omega) = h$, then

$$L_h(jh) = \frac{h}{2\pi} \int_{-\pi/h}^{\pi/h} e^{i\omega jh}\, d\omega = \delta_{j,0}$$

by orthogonality.

Reverse. If $L_h(jh) = \delta_{j,0}$ for all j, all nonzero Fourier coefficients of S vanish, forcing $S = h$.

B.1.2 From kernel composition to equation (4)

Write $L_h(x) = \sum_j c_j\, K_\gamma(x - jh)$. Fourier-transforming gives

$$\widehat{L}_h(\omega) = \widehat{K}_\gamma(\omega)\, \widehat{C}_h(\omega), \qquad \widehat{C}_h(\omega) := \sum_j c_j\, e^{-ijh\omega}.$$

Because $\widehat{C}_h$ is $2\pi/h$-periodic,

$$\sum_{m\in\mathbb{Z}} \widehat{L}_h\Big(\omega + \tfrac{2\pi m}{h}\Big) = \widehat{C}_h(\omega) \sum_{m\in\mathbb{Z}} \widehat{K}_\gamma\Big(\omega + \tfrac{2\pi m}{h}\Big) = \widehat{C}_h(\omega)\, D_h(\omega).$$

Plugging into (3) yields $h = \widehat{C}_h(\omega)\, D_h(\omega)$, so

$$\widehat{C}_h(\omega) = \frac{h}{D_h(\omega)}.$$

B.1.3 Recovering the coefficients c_j

Rescale $\theta := h\omega$ and define

$$\widetilde{C}_\lambda(\theta) := \widehat{C}_h(\theta/h) = \frac{h}{\widetilde{D}_\lambda(\theta)}.$$

This is 2π-periodic with Fourier series $\widetilde{C}_\lambda(\theta) = \sum_j c_j\, e^{-ij\theta}$. The Fourier coefficients of $\widetilde{C}_\lambda$ are the kernel-expansion weights c_j.

Remark. The rescaled normalizer $\widetilde{D}_\lambda(\theta) = D_h(\theta/h)$ depends on γ and h only through $\lambda = \gamma h$, which is why λ is the natural dimensionless parameter.

B.2 Cardinal Coefficient Computation

The cardinal coefficients $\{c_j\}_{|j|\le K_c}$ can be computed by two equivalent routes. The Fourier normalization of Section 3.1 provides the conceptual derivation; the Toeplitz system described below is the practical finite-stencil implementation used in the code.

Toeplitz route (primary implementation). The truncated cardinal condition $\sum_{|j|\le K_c} c_j\, K_\gamma((k-j)h) = \delta_{k,0}$ for $|k| \le K_c$ is a symmetric Toeplitz system $Tc = e_0$ with entries $T_{k,j} = h\, K_\gamma((k-j)h)$. This is the route used in our implementation and is particularly natural in extended-precision arithmetic, since the system matrix depends only on (λ, K_c) and can be assembled and factored once per parameter set.

DFT route (conceptual). Alternatively, evaluate the rescaled Fourier character $\widetilde{C}_\lambda(\theta) = h/\widetilde{D}_\lambda(\theta)$ on an M-point DFT grid ($M > 2K_c + 1$) and invert. The DFT returns aliased coefficients $\widetilde{c}_j^{(M)} = c_j + \sum_{l\ne 0} c_{j+lM}$; by Lemma 2 ($|c_j| \le C_c(\lambda)\, e^{-\sigma|j|}$), this aliasing error can be driven below machine precision by choosing M moderately larger than $2K_c + 1$.

Both routes produce identical coefficients in exact arithmetic.

B.3 Kernel Network Re-indexing

Substituting (5) into (6) and setting $m = k + j$ gives

$$(Q_{h,R}^{(K_c)} f)(x) = \sum_{k=-R}^{N+R} f(x_k) \sum_{|j|\le K_c} c_j\, K_\gamma(x - x_k - jh) = \sum_{m=-R}^{N+R} a[m]\, K_\gamma(x - x_m),$$

where $a[m] = \sum_{|j|\le K_c} c_j\, f(x_{m-j})$ is a discrete convolution of width $2K_c + 1$. This requires samples on the extended index set $\{-R - K_c, \dots, N + R + K_c\}$.

B.4 Activation–Kernel Verification

For $\psi = \tanh$, the derivative-kernel relation $K = \psi' = \mathrm{sech}^2$ satisfies all conditions (K1–K4) as verified in Appendix A.1.3: nonzero mass ($\widehat{K}(0) = 2$), exponential spatial decay with $(A_K, a_K) = (4, 2)$, positive Fourier transform, and a diagonal-dominance condition ensuring a zero-free strip. For $r = 1$, the integration polynomial in (11) reduces to a single bias term.

In principle, any smooth activation whose r-th derivative satisfies K1–K4 yields a valid construction. However, extending the verification beyond $\tanh$ requires care: $\mathrm{GELU}''(x) = \phi(x)(2 - x^2)$ changes sign, and $\mathrm{Softplus}'(x) = \sigma(x)$ does not decay as $x \to +\infty$, violating K4. We leave a systematic treatment to future work; all experiments use $\tanh$.

C Width–Precision Scaling

C.1 The Viable Regime for λ

For a fixed target tolerance ε, the bandwidth λ must satisfy competing constraints.

Aliasing (upper bound). $C_2\, e^{-c_2/\lambda^p} \le \varepsilon$ requires

$$\lambda \le \lambda_{\max}(\varepsilon) := \Big(\frac{c_2}{\log(C_2/\varepsilon)}\Big)^{1/p}.$$

Conditioning (lower bound). The prefactors $C_i(\lambda)$ diverge as $\lambda \to 0$. For $K = \mathrm{sech}^2$, the spatial decay of the cardinal function (Lemma 4) requires $a_K\lambda > \sigma$, giving $\lambda > \sigma/a_K$. The diagonal-dominance condition (Proposition 4) further constrains λ from below.

The viable regime $[\lambda_{\min}, \lambda_{\max}(\varepsilon)]$ is nonempty for all ε above a kernel-dependent threshold. This threshold arises because $\lambda_{\max}(\varepsilon) \to 0$ as $\varepsilon \to 0$ while $\lambda_{\min}$ is fixed by conditioning. The interval shrinks slowly (as $1/\log(1/\varepsilon)$) with increasing precision.

Practical selection. The optimal λ^* within the viable regime balances the four error terms in Theorem 1. Smaller λ^* reduces aliasing but worsens conditioning and, in finite-precision arithmetic, increases cancellation in the cardinal-coefficient convolution (Appendix D.6). The values used in our experiments are reported in Appendix D.2.

C.2 Proof of Corollary 1

Fix ε and choose λ^* in the viable regime so that the aliasing term satisfies $C_2\, e^{-c_2/(\lambda^*)^p} =: \delta \le \varepsilon$. Given width W, allocate

$$N = \lfloor \beta W \rfloor, \quad R = \Big\lfloor \frac{(1-\beta)W}{2} \Big\rfloor, \quad K_c = R$$

for fixed $\beta \in (0,1)$. Then $h = 2/N \le 4/(\beta W)$, and:

- *Resolution:* $C_1(\lambda^*)\, e^{-c_1 N/2} \le C_1(\lambda^*)\, e^{-c_1 \beta W/4}$.
- *Halo:* $C_3(\lambda^*)\, e^{-c_3 \lambda^* R} \le C_3(\lambda^*)\, e^{-c_3 \lambda^* (1-\beta) W/4}$.
- *Stencil:* $C_4(\lambda^*)\, e^{-c_4 \lambda^* R} \le C_4(\lambda^*)\, e^{-c_4 \lambda^* (1-\beta) W/4}$.

Setting

$$\alpha := \min\Big(\frac{c_1 \beta}{4},\ \frac{c_3 \lambda^* (1-\beta)}{4},\ \frac{c_4 \lambda^* (1-\beta)}{4}\Big), \qquad A := C_1 + C_3 + C_4 + \delta$$

yields $\|f - \widetilde{f}_W\| \le A\, e^{-\alpha W}$ for W large enough that $N \ge 2$ and $R \ge 1$. □

Remark. The constant A depends on λ^* and hence (weakly) on ε. The proportion β can be optimized: $\beta^* = c_3 \lambda^* / (c_1 + c_3 \lambda^*)$.

C.3 Weight Magnitude Scaling

With λ^* fixed, $\gamma = \lambda^* N/2$.

Inner-layer weights. $\gamma = O(W) = O(\log(1/\varepsilon))$ by Corollary 1.

Outer-layer weights.

$$|a[m]| \le \|f'\|_\infty \sum_{|j| \le K_c} |c_j| \le \|f'\|_\infty\, C_c(\lambda^*)\, \coth(\sigma/2) = O(1)$$

by Lemma 2. For $r = 1$ (tanh), the outer weights $w[m] = a[m]$ are $O(1)$.

Maximum weight. $\|W\|_{\max} = \gamma = O(\log(1/\varepsilon))$. Concretely, for $\varepsilon = 10^{-15}$: our construction gives $\gamma \sim 35$, versus $\sim 10^{15}$ for Cybenko (1989) and $\sim 10^{15/k}$ for Mhaskar (1993).

D Practical Implementation Details

This appendix provides the engineering details needed to reproduce the QI–MLP construction from Section 3.4.

D.1 Parameter Roles

The construction has four numerical design parameters:

Parameter	Role	Error term controlled
$\lambda = \gamma h$	Dimensionless bandwidth	Aliasing ($e^{-c_2/\lambda}$) vs. conditioning
N	Number of grid intervals ($N{+}1$ in-domain nodes)	Resolution ($e^{-c_1 N/2}$)
R	Halo radius (nodes outside Ω)	Boundary truncation ($e^{-c_3 \lambda R}$)
K_c	Stencil half-width	Cardinal function truncation ($e^{-c_4 \lambda K_c}$)

The total MLP width is $W = N + 2R + 1$.

D.2 Precision Regimes

Two precision modes are used, both producing valid fp64 model weights:

	fp64 path	**Extended-precision path**
λ^*	0.30	0.25
K_c	160	160
R (halo)	$\max(59,\ \lceil 0.4N \rceil)$	$\max(70,\ \lceil 0.4N \rceil)$
Extended digits	—	30
Typical L^∞	$\sim 10^{-12}$	$\sim 3 \times 10^{-15}$

fp64 construction. All arithmetic (Toeplitz solve, convolution, bias computation) is performed in fp64. This is fast but the convolution step involves cancellation among large alternating-sign cardinal coefficients ($|c_0|$ can reach $O(10^2)$ at $\lambda = 0.30$), producing an effective precision floor around 10^{-12}.

Extended-precision construction. The Toeplitz solve and convolution are performed in extended precision (e.g., 30 decimal digits via mpmath), then the resulting MLP coefficients are rounded to fp64. This is an offline, target-independent precomputation—it does not violate the fp64 model assumption. The analogy is exact: `numpy.pi` is derived from extended-precision algorithms and stored as a single fp64 constant; nobody considers this a violation of fp64 arithmetic. What matters is that the coefficients (a_m, b, centers) loaded into the PyTorch model are valid `float64` values. Whether they were computed by hand, in fp64, or in mpmath is irrelevant to the training and evaluation regime.

The extended-precision route is the canonical ground truth for "what is the best fp64 coefficient set we can construct?" The fp64 route is a fast approximation of that ground truth, accurate enough for most training experiments.

D.3 Caching

The cardinal coefficients c_j depend only on (λ, K_c, precision mode)—crucially, *not* on the target function. We cache them to disk after the first computation. This is essential for the extended-precision path, where the Toeplitz solve dominates runtime ($\sim$55 s cold) while the per-target convolution takes $\sim$0.1 s and can be repeated cheaply.

For a typical experiment touching 8 targets at 5 widths with 5 seeds each, the cold extended-precision cost is 5×55 s ≈ 5 min (one solve per width); all subsequent calls complete in $\sim$0.25 s from cache. The cache pays for itself on the first experiment.

D.4 Precision Recommendations per Experiment

Experiment	Precision	Rationale
Numerics sanity	both	explicitly compares the two paths
λ-tradeoff sweep	fp64	sweeps many params; speed matters
Basin stability	extended	QI solution is the reference point
Geometry ladder	extended	measures precision loss vs. construction
Hessian analysis	extended	Hessian at true QI solution
Feature-matrix conditioning	fp64	studies feature matrix, not coefficients
Objective mismatch	fp64	training runs
Noise sensitivity	fp64	fast sweeps
Reparameterization	fp64	training comparisons

Rule of thumb. Use extended precision whenever the construction is a *fixed reference* being compared against. Use fp64 whenever the task involves training, sweeping, or initializing. Both paths produce valid fp64 coefficients that initialize a fp64 PyTorch model.

D.5 Halo and Stencil Sizing

The halo size R must be large enough that the halo-truncation term $e^{-c_3 \lambda R}$ falls below the target precision. Setting $R \geq \lceil 35/(2\lambda) \rceil$ ensures the halo floor is below $e^{-35} \approx 6 \times 10^{-16}$, i.e., below fp64 machine epsilon. Additionally, R scales with N to maintain coverage as the grid refines (empirically $R \approx 0.4\,N$ suffices); the effective rule is $R = \max(\lceil 35/(2\lambda) \rceil,\ \lceil 0.4\,N \rceil)$.

The stencil half-width K_c must be large enough that the cardinal-coefficient tails are negligible. The simplified decay rate $e^{-2\lambda K_c}$ underestimates the required stencil because the actual singularities of $h/D_h(\omega)$ in the complex plane produce slower decay. We fix $K_c = 160$, which gives edge-coefficient magnitudes below 10^{-18} at $\lambda = 0.25$.

D.6 Numerical Stabilization

The convolution $a[m] = \sum_{|j|\le K_c} c_j\, f'(x_{m-j})$ is the numerically most delicate step. The cardinal coefficients alternate in sign with magnitudes that can be much larger than the final output, so naive summation accumulates rounding error. Two mitigations are used:

Compensated (Kahan) summation. Reduces the effective rounding error from $O(\sqrt{n}\,\varepsilon_{\mathrm{mach}})$ to $O(\varepsilon_{\mathrm{mach}})$ per element. This is the default in our fp64 evaluation path.

Extended-precision precomputation. Computing the cardinal coefficients in extended precision and then forming the convolution in extended precision before rounding to fp64 eliminates the cancellation issue entirely. This is the route used for machine-precision baselines.

D.7 Known Limitations

- The fp64 construction path plateaus at $\sim 10^{-12}$ due to convolution cancellation; this is unfixable without extended-precision arithmetic.
- The extended-precision path costs ∼55 s per cold call; this is amortized by the cache but remains the dominant cost for new parameter configurations.
- Targets with narrow analyticity strips (e.g., steep sigmoids, high-frequency oscillations) require larger N to converge. This is a *resolution* limitation of the grid, independent of the construction arithmetic.
- We have not implemented per-width λ-sweeps (as done in some prior work); a small sweep over $\lambda \in [0.25, 0.45]$ per width could further improve the fp64 path.